

Fiori Elements 開發課程 5：Controller Extension

環境：BTP ABAP Environment Trial (沿用 fe01 的 `fe01_connection_test` 專案)

Lecture

這一課要解決的問題

fe04 的 Custom Page 是「整頁重寫」——`NoteObjectPage` 從此變成一個完全自己刻的 View / Controller，範本自動生成的 Create / Edit / Delete / Draft 工具列全部消失，換來的是完全的排版自由度。但大多數客製化需求沒有這麼激烈：List Report / Object Page 的範本本身已經做得很好（篩選、排序、變體管理、Draft 全套都有），只是想在某個時間點「插一段自己的邏輯」，或加一顆按鈕——這種情境如果也用 Custom Page 整頁重寫，等於把範本已經做好的一大堆東西全部重做一次，浪費。

Controller Extension 就是官方為這種「保留範本、只加邏輯」情境設計的機制：不動範本的 View，只在 `manifest.json` 裡登記一個你自己寫的 Controller 類別，掛到範本既有 Controller 的生命週期上——範本該做的事（渲染、資料綁定、CRUD 工具列）繼續做，你的程式碼在框架指定的時間點被額外呼叫一次。

官方文件先查證：這是好幾種不同機制的其中一種

查 `SAP-docs/sapui5 ( configuration-in-the-manifest-json )` 確認，SAPUI5 的通用擴充機制有一張對照表，`sap.ui5/extends/extensions` 底下總共有五種：

擴充類型	manifest 區段	用途
View Extension (Extension Point)	<code>sap.ui.viewExtensions</code>	在 View 裡預留的位置插入自訂內容
Controller Extension	<code>sap.ui.controllerExtensions</code>	不取代既有 Controller ，只是掛上額外的邏輯 / 生命週期 Hook
View Modification	<code>sap.ui.viewModifications</code>	改既有控制項的簡單屬性（目前只能改 <code>visible</code> ）
View Replacement	<code>sap.ui.viewReplacements</code>	整個 View 換掉（跟 fe04 的 Custom Page 概念類似，但這是舊式 Freestyle App 的機制）
Controller Replacement	<code>sap.ui.controllerReplacements</code>	整個 Controller 換掉

這一課只做 Controller Extension——它是這五種裡「侵入性最小、最常用」的一種。另外，Fiori Elements 還有一個專屬於範本頁面、不算在這張表裡的獨立機制：`content.header.actions / controlConfiguration...actions` 這種「Custom Action」——只是幫按鈕的 `press` 指到一個單純的 JS 函式，完全不用宣告 `sap.ui.controllerExtensions`。這一課會把兩者都做一遍，順便體會「同樣是加自訂邏輯，框架卻準備了兩種輕重不同的路」。

目標鎖定 List Report：Object Page 已經在 fe04 變成 Custom Page

Controller Extension 掛的是範本自己的 **Controller**（`sap.fe.templates.ListReport.ListReportController / sap.fe.templates.ObjectPage.ObjectPageController`）——`NoteObjectPage` 在 fe04 已經換成 `sap.fe.core.fpm`（Custom Page，整頁自己刻的 View / Controller），根本沒有範本 Controller 可以掛，要加邏輯直接寫在 `NoteDetail.controller.js` 裡就好，用不上 Controller Extension。`NoteList`（List Report）維持原樣（`sap.fe.templates.ListReport`），這一課全部的東西都掛在它身上。

實作一：Controller Extension——`routing.onAfterBinding` 動態訊息條

查證 `sap-help ( Adding a Custom Message Strip to List Report Page, Analytical List Page and Object Page )` 確認：List Report 的 Controller Extension 可以用 `this.base.getExtensionAPI().setCustomMessage(...)` 在表格上方顯示一段自訂訊息，範例就是掛在 `routing.onAfterBinding` 這個 Hook 裡——這是 FE 框架專屬的生命週期事件。

⚠️ ⚠️ 實測結果跟原本的假設不符，這裡先講清楚查證結論，後面「驗證方式」會附完整的實測過程：一開始看官方文件的命名（`onAfterBinding`）跟範例，直覺會以為「List Report 每次重新綁定資料（第一次載入、按 Go、切換篩選條件）都會呼叫一次」——**實際測試發現不是這樣**：這個 Hook 只在這次頁面停留期間、真正建立新綁定的那一刻觸發一次（也就是路由 / 導覽進到這個頁面、第一次把資料綁上表格的當下），同一頁停留期間按 Go 重新查詢，不會讓它再觸發第二次。名稱裡的「routing」是關鍵線索——這個 Hook 綁的是「路由匹配、頁面被導覽進來」這件事，不是「表格資料被重新整理」這件事，兩者容易被誤會成同一回事，但實測證實不是。想要「每次查詢都執行」的效果，這個 Hook 不是正確的掛勾點，需要另外查證 Table Building Block 的其他 API（例如資料請求相關的 Hook），這一課先如實記錄查證出來的真實行為，不深入挖這個延伸問題。

`manifest.json` 改動（`sap.ui5` 底下新增 `extends` 節點）：

```
"sap.ui5": {
  "flexEnabled": true,
  "extends": {
    "extensions": {
      "sap.ui.controllerExtensions": {
        "sap.fe.templates.ListReport.ListReportController": {
```

```

        "controllerName": "fe01connectiontest.ext.controller.NoteListExt"
    }
}
},
...
}

```

新增的 **Controller** ([webapp/ext/controller/NoteListExt.controller.js](#)) :

```

sap.ui.define(
    ["sap/ui/core/mvc/ControllerExtension", "sap/ui/core/library"],
    function (ControllerExtension, coreLibrary) {
        "use strict";

        var MessageType = coreLibrary.MessageType;

        return ControllerExtension.extend("fe01connectiontest.ext.controller.NoteListExt", {
            override: {
                onInit: function () {
                    // 一般 UI5 controller lifecycle hook · 跟框架自己的 onInit 一起被呼叫 · 不會互相取代
                    console.log("[NoteListExt] onInit fired");
                },
                routing: {
                    // FE 框架專屬的 lifecycle hook · 只在頁面停留期間「真正建立新綁定」時觸發一次 ( 導覽進頁面、首次把資料綁上表格 ) ;

                    // 實測確認：同一頁停留期間按 Go 重新查詢不會再次觸發 · 這個 Hook 綁的是「路由/導覽」不是「表格資料重新整理」
                    onAfterBinding: function (oBindingContext, mParameters) {
                        var extensionAPI = this.base.getExtensionAPI();
                        var sNow = new Date().toLocaleTimeString();

                        extensionAPI.setCustomMessage({
                            message: "資料於 " + sNow + " 重新載入完成 ( 這則訊息由 Controller Extension 的 routing.onAfterBinding 動態產生 )",
                            type: MessageType.Information
                        });
                    }
                }
            }
        });
    }
);

```

override 結構要注意的地方：**onInit** 直接放在 **override** 底下——這是一般 **UI5 Controller** 本來就有的生命週期方法 (**onInit** / **onExit** / **onBeforeRendering** / **onAfterRendering**) · **Controller Extension** 可以直接覆寫；但 **routing** / **editFlow** 這幾個是**FE** 框架自己額外定義的命名空間 · 要包一層才找得到框架會呼叫的 Hook (**routing.onAfterBinding**、**editFlow.onBeforeCreate**、**editFlow.onAfterSave** 等) 。兩種 Hook 混在同一個 **override** 物件裡 · 只是巢狀層級不同 · 不要搞混。

this.base.getExtensionAPI() 是官方唯一支援的存取管道：查證的兩份文件 ([Adding Custom Actions Using Extension Points / Fiori Elements Integration OData V4](#)) 都強調同一件事——「Use app extensions with caution and only if you cannot produce the required behavior by other means」「use only the extensionAPI of SAP Fiori elements. Don't access or manipulate controls, properties, models, or other internal objects created by the SAP Fiori elements framework.」——**this.base** 是框架注入的、指向被擴充的原始 **Controller** 的參照 · **getExtensionAPI()** 回傳的是框架刻意包裝過的公開介面 · 不要嘗試繞過它直接抓框架內部的 **View / Control** 。

實作二：Custom Action——List Report 全域按鈕 · 不用宣告 **Controller Extension**

查證 **sap-help** ([Adding Custom Actions Using Extension Points](#) · **OData V4** 版本) 確認：如果只是想加一顆按鈕 · 有一條更輕量的路——在 **manifest.json** 的 **content.header.actions** 底下指定 **press** 指到一個單純的 **JS** 模組函式 · 完全不用碰 **sap.ui.controllerExtensions** 。

manifest.json 改動 (**NoteList** target 的 **options.settings** · 跟 **navigation** 平行) :

```

"content": {
    "header": {
        "actions": {
            "showInfoAction": {

```

```
        "press": "fe01connectiontest.ext.CustomActions.showInfo",
        "visible": true,
        "text": "{i18n>showInfoActionText}"
    }
}
}
```

新增的 JS 模組 (`webapp/ext/CustomActions.js` · 注意檔名沒有 `.controller.js` 後綴——它根本不是 Controller · 只是一個被動呼叫的函式庫) :

```
sap.ui.define(["sap/m/MessageToast"], function (MessageToast) {
    "use strict";

    return {
        showInfo: function (oContext, aSelectedContexts) {
            MessageToast.show("這是 List Report 全域 Custom Action · 透過 Extension Point 加的按鈕 · 不需要完整的 Controller Extension 註冊");
        }
    };
});
```

`press` 函式的簽章固定是 (`oContext` , `aSelectedContexts`)——`oContext` 是目前的 Binding Context · `aSelectedContexts` 是表格裡目前被勾選的資料列 (陣列) 。這一課的按鈕是「List Report 全域動作」 (不需要先選資料列) · 所以兩個參數都用不到 · 但簽章統一是這樣 · Table Toolbar Action / Section Action 用同一套函式簽章。

兩種機制的取舍

	Controller Extension	Custom Action
manifest 要註冊什麼	<code>sap.ui.controllerExtensions</code> (整個 Controller 類別)	直接在 <code>actions</code> 底下指一個函式路徑
程式碼型態	Class (<code>ControllerExtension.extend(...)</code>)	單純的 Function (<code>sap.ui.define([], function(){ return {...}; })</code>)
能力範圍	框架生命週期任何 Hook (<code>onInit</code> / <code>routing.*</code> / <code>editFlow.*</code>) · 加按鈕也要用它	只能回應「使用者按下這顆按鈕」這一件事
適合情境	需要在特定時機自動觸發邏輯 (資料重新載入後、儲存後.....)	只是想加一顆按鈕 · 邏輯完全由使用者主動觸發

這一課刻意兩個都做 · 是因為光看官方文件很容易誤以為「加自訂 JS 邏輯」只有一種做法——實際上框架依照「要不要掛生命週期」把它拆成了輕重不同的兩條路 · 選錯 (例如為了一顆按鈕去宣告一整個 Controller Extension) 不會出錯 · 但是不必要的重量級寫法。

學習目標

- 知道 SAPUI5 的擴充機制有五種 (`sap.ui.viewExtensions` / `controllerExtensions` / `viewModifications` / `viewReplacements` / `controllerReplacements`) · Controller Extension 只是其中一種 · 且是侵入性最小的一種
- 知道 Controller Extension 掛的是範本自己的 **Controller** (`sap.fe.templates.ListReport.ListReportController`) · 已經被 fe04 換成 Custom Page 的頁面沒有範本 Controller 可掛 · 這個機制對它沒有意義
- 能寫出 `sap.ui.controllerExtensions` 的 manifest 註冊語法與對應的 `ControllerExtension.extend(...)` 程式碼
- 知道 `override` 底下 · 一般 UI5 生命週期方法 (`onInit` 等) 直接放 · FE 框架專屬的 Hook (`routing.onAfterBinding` 等) 要包一層命名空間
- 知道 `this.base.getExtensionAPI()` 是官方唯一支援的存取管道 · 不應該試圖繞過它直接操作框架內部物件
- 知道「加一顆按鈕」有更輕量的 Custom Action 機制 (`content.header.actions` + 單純 JS 函式) · 不需要動用完整的 Controller Extension

物件清單

延續 fe01 ~ fe04 · 繼續在同一個前端專案上疊加 · 沒有新增任何 ABAP 物件 :

檔案	這一課的修改
<code>fe01_connection_test/webapp/manifest.json</code>	新增 <code>sap.ui5.extends.extensions.sap.ui.controllerExtensions</code> (掛 <code>NoteListExt</code>) ; <code>NoteList</code> target 新增 <code>content.header.actions.showInfoAction</code>
<code>fe01_connection_test/webapp/ext/controller/NoteListExt.controller.js</code>	新增 · List Report 的 Controller Extension (<code>routing.onAfterBinding</code> 動態訊息條)

檔案	這一課的修改
<code>fe01_connection_test/webapp/ext/CustomActions.js</code>	新增 · List Report 全域 Custom Action 的 press handler
<code>fe01_connection_test/webapp/i18n/i18n.properties</code>	新增 <code>showInfoActionText</code>

動手練習

輪到你了：

- 幫 `NoteListExt.controller.js` 的 `override` 加一個 `editFlow.onBeforeCreate` Hook (查一下 EditFlow API 有沒有這個 Hook、簽章長什麼樣子)，在裡面用 `console.log` 印一行訊息——體驗一下「按 Create 按鈕」這個動作也是框架生命週期的一部分，能被 Controller Extension 攔到
- 把 `CustomActions.js` 的 `showInfo` 函式改成讀取 `aSelectedContexts.length`，訊息文字改成「目前選取了 N 筆資料」——體驗一下 Table Toolbar Action (而非 List Report 全域 Action) 情境下 `aSelectedContexts` 會有內容的陣列，跟這一課全域按鈕用不到它的情境做對照 (提示：這個修改可以直接測，因為 fe03 已經把 List Report 表格的 `selectionMode` 設成 `Multi`)
- 想一想 (不用真的做)：如果要在 `NoteDetail` (fe04 的 Custom Page) 裡也想要「資料重新綁定時顯示一段動態訊息」，你會怎麼做？ (提示：Custom Page 沒有範本 Controller 可以掛 Controller Extension，但你自己的 `NoteDetail.controller.js` 本來就是完全你自己寫的程式碼——想一想直接在裡面寫會是什麼樣子，跟這一課「掛外部邏輯到別人的 Controller 上」的思維有什麼本質差異)

驗證方式

`npm start` 本機執行 + 瀏覽器截圖確認。這一課的驗證過程本身就發現了一個跟原始假設不符的行為，完整記錄如下：

- 打開 List Report (`NoteList`)，確認頁面右上角 (Share 按鈕附近) 多了一顆「顯示頁面資訊」按鈕，點下去跳出 MessageToast——已截圖確認，按鈕存在、點擊有反應。
- 訊息條的實測過程 (已截圖確認，共測了兩輪)：
 - 第一輪 (登入後直接進頁面，未特別按過 Go)：表格上方直接出現藍色 **Information** 訊息條，文字包含時間 (例如「資料於 上午10:04:13 重新載入完成.....」)；接著按 **Go** 重新查詢，**訊息條的時間沒有更新**。
 - 第二輪 (把整個瀏覽器頁面 F5 重新整理，排除「同一 Session 殘留」的干擾)：重新整理完**沒有**訊息條；按一次 **Go**，訊息條才出現、時間是當下的最新時間；**再按一次 Go**，訊息條**沒有更新**，時間跟第一次按 Go 時相同。
 - 結論：`routing.onAfterBinding` 在這一頁的停留期間**只觸發一次** (在真正建立新綁定的那一刻——可能是導覽進頁面，也可能是頁面重新整理後第一次成功把資料綁上表格)，之後同頁面不管按幾次 Go，都不會再觸發第二次。這跟原本看官方範例直覺以為的「每次查詢都會更新」不符，已回頭訂正上面 Lecture 段落與程式碼註解。
- 打開瀏覽器開發者工具的 Console 分頁，重新整理頁面，應該能看到印出 `[NoteListExt] onInit fired` (這一項屬於一般 UI5 生命週期，不受上面 `onAfterBinding` 的限制，重新整理理論上都會觸發，未逐次截圖驗證，讀者可自行確認)。

思考題

- `routing.onAfterBinding` 這個 Hook 名稱裡的「routing」，其實暗示它是掛在框架的路由 (Route/Target) 機制上，不是單純的「Controller 生命週期」。實測已經證實：同一頁停留期間按 Go 重新查詢，不會讓這個 Hook 再次觸發——換句話說，「篩選條件改變、重新查詢」在這個 Hook 的定義裡**不算**一次新的路由匹配，只有「真正導覽/重新整理進入這個頁面」才算。想一想：如果你的需求是「使用者每次按 Go 查詢，都要跑一段自訂邏輯」，`routing.onAfterBinding` 顯然不是對的掛勾點，你會怎麼查證框架有沒有提供對應「表格資料重新整理」這個時機的 Hook？ (提示：想一想這一課提過的 Table Building Block、`sap.fe.macros.table` 相關 API，這是一個「查證方法」的練習，不要求真的做出來)
- 這一課的 Custom Action (`showInfo`) 跟 fe04 練習題裡「在 Custom Page 加一顆按鈕」，兩者都是「按下去執行一段 JS」，但背後的程式碼型態完全不同 (一個是單純函式、一個是 View 裡的 `press=".onXxx"` 綁到 Controller 方法)。如果今天的需求是「這顆按鈕要能存取目前使用者在篩選列打的查詢條件」，你覺得哪一種寫法比較容易做到？為什麼？
- 這一課完全沒有用到 `sap.ui.viewExtensions` (View Extension / Extension Point，五種機制列表裡的第一種)。查一下官方文件對它的定位——它跟 Controller Extension 通常是「搭配使用」還是「二選一」的關係？ (提示：想一想「View 裡要有一個預留位置」跟「Controller 要有邏輯」，這兩件事是不是經常同時發生)

答案

見 `fe01_connection_test/webapp/manifest.json` (`sap.ui5.extends.extensions`、`NoteList` target 的 `content.header.actions`)、`fe01_connection_test/webapp/ext/controller/NoteListExt.controller.js`、`fe01_connection_test/webapp/ext/CustomActions.js`、`fe01_connection_test/webapp/i18n/i18n.properties` (`showInfoActionText`)。沒有新增或修改任何 ABAP 物件，也沒有新增前端專案——延續 fe01 ~ fe04 對同一個 `fe01_connection_test` 專案的疊加修改。